

Fitcheck

Complete the Look Recommendation System



Soni Toker, Coral Shmuelovitz, Odeya Lapian

תוכן עניינים:

3	מהי האפליקציה?
4	Dataset
5	עיבוד הנתונים
6	המודל
6	שליחת התמונה דרך המודל
6	חישוב מרחקים בין תמונות
7	בניית המודל ברמת הקוד
11	בניית אתר דמה
15	האפליקציה הסופית
18	FITCHECK BOT

מהי האפליקציה?

אפליקציה חדשנית המאפשרת למשתמשים להעלות פריטי בגדים קיימים ולקבל המלצות לפריטים משלימים שיתאימו לפריט שהועלה. האפליקציה מציגה מסך בית עם אפשרות להעלות פריט לבוש (למשל חולצה). לאחר ההעלאה, המשתמש מקבל רשימה של פריטים משלימים כמו מכנסיים, נעליים או אקססוריז שמתאימים לסגנון ולצבע הפריט שהועלה.

השימושים המרכזיים :

- התאמת בגדים לפי סגנון, צבעים והעדפות אישיות.
- חוויית קנייה מותאמת אישית ומהירה .
- כלי עזר לאנשים המעוניינים לשדרג את המלתחה שלהם בקלות .

היתרונות :

- נוחות וזמינות: במקום להתייעץ עם סטייליסטית, האפליקציה מספקת פתרונות מידיים וזמינים.
- חיסכון בזמן: מציאת התאמות מדויקות בלחיצת כפתור, ללא צורך בחיפושים ארוכים.
- חדשנות טכנולוגית: שימוש בטכנולוגיות מתקדמות כמו זיהוי תמונה ולמידת מכונה להתאמות מדויקות וסגנוניות.

Dataset

נעזרנו בדאטא סט המוכר Complete the Look Dataset שנלקח מהמאגר של Kaggle. הדאטאסט כולל אלפי תמונות של פריטים בקטגוריות שונות כמו בגדים, נעליים, אביזרים ותכשיטים. כל תמונה מסווגת לקטגוריה מתאימה ומכילה מידע חזותי עשיר. הדאטה סט מכיל זוגות נתונים של סצנות ומוצרים, יחד עם תיבות גבול למוצרים ומידע קטגוריאל אודותיהם. המוצרים נוצרו על ידי חיתוך מתוך סצנות שונות.

לדוגמא:



הפורמט של הדאטאסט:

Format

The datasets are provided in `tsv` format. We have two different versions:

1. Raw dataset with header (train and test): `(image_signature, bounding_x, bounding_y, bounding_width, bounding_height, label)`
2. Triplet dataset with header (train only): `(image_signature_anchor, bounding_x_anchor, bounding_y_anchor, bounding_width_anchor, bounding_height_anchor, label, image_signature_pos, bounding_x_pos, bounding_y_pos, bounding_width_pos, bounding_height_pos, label, image_signature_neg, bounding_x_neg, bounding_y_neg, bounding_width_neg, bounding_height_neg, label)`

Each image is a collage outfit and each bounding_box represents an item. Multiple rows with the same image_signature contain items from the same outfit.

דוגמא מהקובץ של הדאטאסט:

image_signature	bounding_x	bounding_y	bounding_width	bounding_height	label
00015b9d0c08164ed3b5dbb4613bb143	0.0068093464	0.01915935	0.4946373	0.31540385	Shirts & Tops
00015b9d0c08164ed3b5dbb4613bb143	0.019453462	0.33092913	0.49747342	0.3255237	Shorts
00015b9d0c08164ed3b5dbb4613bb143	0.46410304	0.36533436	0.2773865	0.24736455	Hats
00015b9d0c08164ed3b5dbb4613bb143	0.053017423	0.6521988	0.37513447	0.32219654	Shoes
00015b9d0c08164ed3b5dbb4613bb143	0.5054301	0.70862186	0.42420077	0.23256832	Handbags
0036db3d8bca72e539219985a6f2c9bb	0.0	0.18509896	0.3719441	0.47956395	Shirts & Tops
0036db3d8bca72e539219985a6f2c9bb	0.3673023	0.22582057	0.35072178	0.37863812	Shoes
0036db3d8bca72e539219985a6f2c9bb	0.3644814	0.5678833	0.31260628	0.26245987	Jewelry
0036db3d8bca72e539219985a6f2c9bb	0.0	0.6354713	0.40069765	0.2477963	Shorts
0036db3d8bca72e539219985a6f2c9bb	0.7307526	0.18195973	0.23790014	0.47855985	Handbags
0039bbfd97f17c0675324d2f6b650849	0.46210417	0.056790717	0.37592986	0.3778662	Shirts & Tops
0039bbfd97f17c0675324d2f6b650849	0.36126918	0.59026045	0.2812336	0.3176911	Shoes
0039bbfd97f17c0675324d2f6b650849	0.5911352	0.42630273	0.14706361	0.34812826	Pants
0039bbfd97f17c0675324d2f6b650849	0.70274556	0.64167845	0.2659204	0.27946734	Sunglasses
0039bbfd97f17c0675324d2f6b650849	0.26921767	0.042181123	0.25142998	0.4771741	Coats & Jackets
0041c5fd55da1da1fcaaf44207a3fa636	0.623411	0.0	0.34160578	0.36237347	Scarves & Shawls

הקוד שממיר את המזהה של התמונה לתמונה עצמה:

```
def convert_to_url(signature):
    prefix = 'http://i.pinimg.com/400x/%s/%s/%s/%s.jpg'
    return prefix % (signature[0:2], signature[2:4], signature[4:6], signature)
```

עיבוד הנתונים:

כדי להתאים את הדאטה סט לשימוש במודל, התמונות עוברו את תהליך העיבוד הבא :

1. **שינוי גודל התמונות:** כל התמונות הותאמו לגודל אחיד של 256x256 פיקסלים .
2. **המרה לטנסורים:** הנתונים הומרו לטנסורים על מנת שיהיו ניתנים לשימוש במודל PyTorch.
3. **נרמול נתונים:** התמונות נרמלו כדי להתאים למודל ה ResNet.
4. **שימוש ב DataLoader:** הנתונים הוזנו ל DataLoader של PyTorch עבור תהליך חילוך התכונות.

המודל

חקרנו מודלים שונים ובסוף נעזרנו במודל שראינו ב-gitHub:
נסביר על התהליך.

שליחת התמונה דרך המודל:

לאחר עיבוד התמונה, היא נשלחת דרך מודל רשת נוירונית עמוקה שמחולל ייצוגים וקטוריים. המודל מורכב ממספר שכבות של קונבולוציה (CNN) אשר לומדות את מאפייני התמונה באופן הדרגתי.

1. **שכבות קונבולוציה:** בשכבות אלה המודל מזהה תכונות בסיסיות כמו קווים, צורות ורגישויות למאפיינים חזותיים אחרים (צבעים, טקסטורות, וכו').
2. **שכבות Fully Connected:** לאחר השכבות הקונבולוציה, המודל כולל שכבות Fully Connected שמספקות הפקת תכונות.
3. במקום להשתמש במודל כולו הוצאנו את הווקטור מהשכבות הגבוהות של המודל על מנת לקבל את הייצוג הווקטורי של התמונה.

חישוב מרחקים בין תמונות:

לאחר שיצרנו את הווקטור, אנחנו יכולים לבצע חישובים בין התמונות על ידי חישוב מרחקים או דימיון בין הווקטורים על פי מרחק קוסינוס-נמדוד את הדמיון בין וקטור תכונות של שתי תמונות בעזרת חישוב הדמיון הקוסיני אם תמונה A ו-תמונה B מצביעות על דימיון גבוה (הווקטורים שלהן קרובים), נוכל לקבוע שהתמונות תואמות.

בניית המודל ברמת הקוד:

בפרויקט ישנם 2 קבצים מרכזיים שמתמקדים בבניית המודל ואימונו:

Model - בנייה בסיסית של מודל המבוסס על רשת נוירונים.

```
"""
Initializes the compatibility model with a ResNet-18 base model and embedding layers.

Args:
    hidden_dim (int): Dimension of the hidden layer in the embedding layers.
    emb_dim (int): Dimension of the final embedding output.
    dropout (float): Dropout rate for regularization.
"""
def __init__(self, hidden_dim=cfg.HIDDEN_DIM, emb_dim=cfg.EMBEDDING_DIM, dropout=cfg.DROPOUT):
    super(CompatibilityModel, self).__init__()
    # use resnet34 as base model
    self.create_base_model()
    # add 2 layers on top of base model
    self.embedding_layers = nn.Sequential(
        nn.Linear(in_features=512, hidden_dim),
        nn.BatchNorm1d(hidden_dim),
        nn.Dropout(dropout),
        nn.Linear(hidden_dim, emb_dim),
    )
```

הפונקציה מקבלת שלושה פרמטרים עבור הגדרת גודל השכבה בשכבת ההטמעה, הגדרת גודל הווקטור הסופי ושיעור הורדת נוירונים למניעת overfitting. הפונקציה קוראת לבנאי המחלקה לאתחל את הרכיבים הבסיסיים של pytorch לאחר מכן מאתחלת את המודל הבסיסי (ResNet-18).

(1) nn.Sequential - אוסף של שכבות שנבנות ברצף.

(2) nn.Linear(512, hidden_dim) - שכבת Fully Connected (לינארית) שמקבלת 512 נוירונים כקלט מהפלט של (ResNet-18) וממפה אותם למספר מוגדר hidden_dim.

(3) nn.BatchNorm1d(hidden_dim) - מנרמלת את הפלט של השכבה הקודמת כדי לשפר את הביצועים.

(4) nn.Dropout(dropout) - מנטרלת אחוז מסוים של נוירונים אקראיים כדי למנוע overfitting.

(5) nn.Linear(hidden_dim, emb_dim) - שכבה שממפה את הנוירונים מהשכבה הנסתר למספר הנוירונים בהטמעה הסופית (emb_dim).

```

"""
    Sets up the base model by loading a pre-trained ResNet-18 and replacing the final
    fully connected layer with an identity layer.
"""
def create_base_model(self): 1 usage  Bowen Chen *
    self.base_model = models.resnet18(pretrained=True)
    self.base_model.fc = nn.Identity()

"""
    Defines the forward pass of the model. Input is passed through the base model
    and then through the embedding layers.

    Args:
        x (Tensor): Input tensor containing image data.

    Returns:
        Tensor: Embedding output from the model.
"""
def forward(self, x): Bowen Chen
    # pass input through base and embedding layers
    x = self.base_model(x)
    x = self.embedding_layers(x)
    return x

```

הפונקציה הראשונה טוענת את המודל ResNet-18 המאומן מראש ומחליפה את השכבה הסופית בשכבה שמשאירה את הפלט ללא שינוי.

הפונקציה השנייה מעבירה את התמונות דרך ResNet-18 כדי לחלץ תכונות ולאחר מכן מעבירה את הפלט לשכבות ההטמעה ליצירת ווקטור הטמעה. הפונקציה מחזירה את ווקטור ההטמעה שמשמש להערכת התאמה בין פריטים.

training - אימון המודל ע"י חישוב הבדלים בין התכונות של הפריטים בכל שלשה ושמירה בקבצי

epoch.pth

```
# training loop
for e in tqdm(range(num_epochs), desc="Epochs"):

    for i, (anchor, positive, negative) in enumerate(
        tqdm(train_dataloader, desc="Training", leave=False)
    ):
        # set gradient accumulation to 0
        optimizer.zero_grad(set_to_none=True)

        # get triplet loss and back update
        loss = get_triplet_loss(anchor, positive, negative, criterion, model)
        loss.backward()

        # update the weights
        optimizer.step()
```

```
# append batch loss to epoch loss
if i % 100 == 0:
    training_losses.append(loss.cpu().detach().numpy())
    avg_training_losses.append(np.mean(training_losses))
    # get validation loss
    scheduler.step(training_losses[-1])
    model.eval()
    with torch.no_grad():
        try:
            iterator = iter(validation_dataloader)
            anchor_val, positive_val, negative_val = iterator.next()
        except: # if reaches the end, reset
            iterator = iter(validation_dataloader)
            anchor_val, positive_val, negative_val = iterator.next()

        # calculate validation loss and backward pass through the model
        loss_valid = get_triplet_loss(
            anchor_val, positive_val, negative_val, criterion, model
        )
        validation_losses.append(loss_valid.cpu().detach().numpy())
        avg_validation_losses.append(np.mean(validation_losses))
```

```
print(
    "\nAvg Training Loss: {:.4f}, Step Training Loss: {:.4f}, Avg Validation Loss: {:.4f}, Step Validation Loss: {:.4f}\n".format(
        *args: np.mean(training_losses),
        training_losses[-1],
        np.mean(validation_losses),
        validation_losses[-1],
    )
)
```

החלק המרכזי של הפונקציה כולל את לולאת האימון, בה המודל מעדכן את המשקולות שלו בכל אפוק (Epoch). הלולאה מתחילה על ידי חישוב ה-Triplet Loss עבור כל batch בנתוני האימון. ה-Triplet Loss מחושב באמצעות שלושה פריטים: תמונת anchor, תמונת positive (תמונה שמתאימה ל-anchor), ותמונת negative (תמונה שאינה מתאימה ל-anchor). הפונקציה `get_triplet_loss` מקבלת את התמונות הללו ומחזירה את ערך האובדן. לאחר חישוב האובדן, מתבצע תהליך של חישוב גרדיאנטים ובעקבותיו עדכון המשקולות של המודל בעזרת `optimizer.step()`. כל 100 batches האובדן נשמר ברשימה וניתן לו דוח על אובדן האימון הממוצע והאובדן עבור הצעד הנוכחי. בנוסף, כל 100 batches המודל עובר שלב אימות (validation), שבו הוא מחשב את האובדן על סט נתוני הוולידציה. במהלך שלב האימות, המודל מועבר למצב `eval()` כדי למנוע חישוב גרדיאנטים מיותרים. אם לא נשארו נתונים בסט הוולידציה, האיטרציה מחדש מתחילה. בסופו של דבר, כל 100 batches המידע על האובדן מודפס, כך שניתן לעקוב אחרי התקדמות האימון וההתקדמות בשיפור המודל.

בניית אתר דמה:

לאחר אימון המודל רצינו לבדוק כי הוא עובד נכון לפני יצירת האפליקציה ולכן המטרה הבאה שלנו הייתה ליצור אתר דמה שמציג את אופן פעולת המודל כדי לבחון את התוצאות.

על מנת לדעת איך להתעסק עם שרתים התייעצנו עם chat GPT שהביא לנו שלבי עבודה ושלדי קודים ששיפרנו והתאמנו לקוד שלנו:

ChatGPT

Memory Full

אנחנו יוצרות אפליקציה שמטרתה לסרוק פריטי בגדים ועל סמך מודלן מאומן שכבר עובד היא תציע פריטים משלימים מתאימים. כרגע לצורך בדיקת פונקציונאליות בסיסית אנחנו רוצות שהאפליקציה רק תקבל פריט מתוך הדאטא סט (שמחולק לתמונות של פריטים בודדים ותמונות של אוטפיטים שלמים על סמך הפריטים הבודדים הללו) אנחנו רוצות שהאפליקציה תתקשר עם השרת שיכניס את הקלט שהוא העלאת תמונה למודל ויחזיר לאפליקציה פלט של המודל (פריטים משלימים) והאפליקציה תציג זאת בצורת תמונות למשתמש.

אני שולחת לך קבצים רלוונטים מאימון המודל בפייטון- אנחנו לא יודעות כעת איך ליצור את כל הבאקנד והפרונטאנד לאפליקציה ותבנה לי הכל בשלבים ברורים בשבילי שאבין מה קורה אם אתה צריך קוד נוספים שלא שלחתי אז תבקש.

- כדי לבנות אפליקציה שתבצע את מה שתיארת, נפריד את המשימות לשלושה שלבים עיקריים:
1. **יצירת השרת (Backend):** השרת יטפל בתקשורת עם המודל, יקבל תמונות מהאפליקציה, יפעיל את המודל, ויחזיר את תוצאות ההמלצות.
 2. **יצירת האפליקציה (Frontend):** האפליקציה תאפשר למשתמש להעלות תמונה, תשלח את התמונה לשרת, ותציג את התוצאות.
 3. **חיבור בין המודל לשרת ולאפליקציה.**

```

from flask import Flask, request, jsonify
from recommend import recommend_complementary_products

app = Flask(__name__)

@app.route('/recommend', methods=['POST'])
def recommend():
    # קבלת קלט
    data = request.json
    product_id = data.get('product_id')

    # הפעלת הפונקציה של המודל
    recommendations = recommend_complementary_products(product_id)

    # תחזיר תוצאות ב JSON
    return jsonify(recommendations)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

Edit ✎ Copy 📄

שלב 2: יצירת האפליקציה (Frontend)

השתמשי ב- React Native או ב- Flutter ליצירת אפליקציה מותאמת מובייל. כאן יש דוגמת עבודה בסיסית ב- HTML + JavaScript שיכולה לרוץ בדפדפן:

קובץ index.html :

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fashion Recommendations</title>
</head>
<body>
  <h1>Upload an Image</h1>
  <input type="file" id="imageUpload" accept="image/*">
  <button id="submitBtn">Get Recommendations</button>
  <div id="results"></div>

  <script src="script.js"></script>
</body>

```

Edit ✎ Copy 📄



: script.js קובץ

Edit ✎ Copy 📄

```

document.getElementById('submitBtn').addEventListener('click', async () => {
  const productId = 123; // זיהוי לדוגמה של פריט
  const response = await fetch('http://localhost:5000/recommend', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ product_id: productId })
  });

  const recommendations = await response.json();
  const resultsDiv = document.getElementById('results');
  resultsDiv.innerHTML = '';

  recommendations.recommended_compatible_products.forEach(product => {
    const img = document.createElement('img');
    img.src = `path_to_images/${product.image_path}`; // החליף במסלול נכון
    resultsDiv.appendChild(img);
  });
});

```



תוצאות המודל:

Complete the Look Recommendation

[Get Recommendations](#)



Input Product
Type: Handbags
Product ID: 5212

Recommended Compatible Products



Coats & Jackets
Product ID: 10078
Compatibility Score: 91.07%



Shorts
Product ID: 10347
Compatibility Score: 90.64%



Skirts
Product ID: 4546
Compatibility Score: 90.42%



Shirts & Tops
Product ID: 6839
Compatibility Score: 89.51%



Pants
Product ID: 5774
Compatibility Score: 88.82%

Complete the Look Recommendation

[Get Recommendations](#)



Input Product
Type: Hats
Product ID: 8525

Recommended Compatible Products



Handbags
Product ID: 1012
Compatibility Score: 88.22%



Shoes
Product ID: 17496
Compatibility Score: 87.81%



Shirts & Tops
Product ID: 17109
Compatibility Score: 87.56%



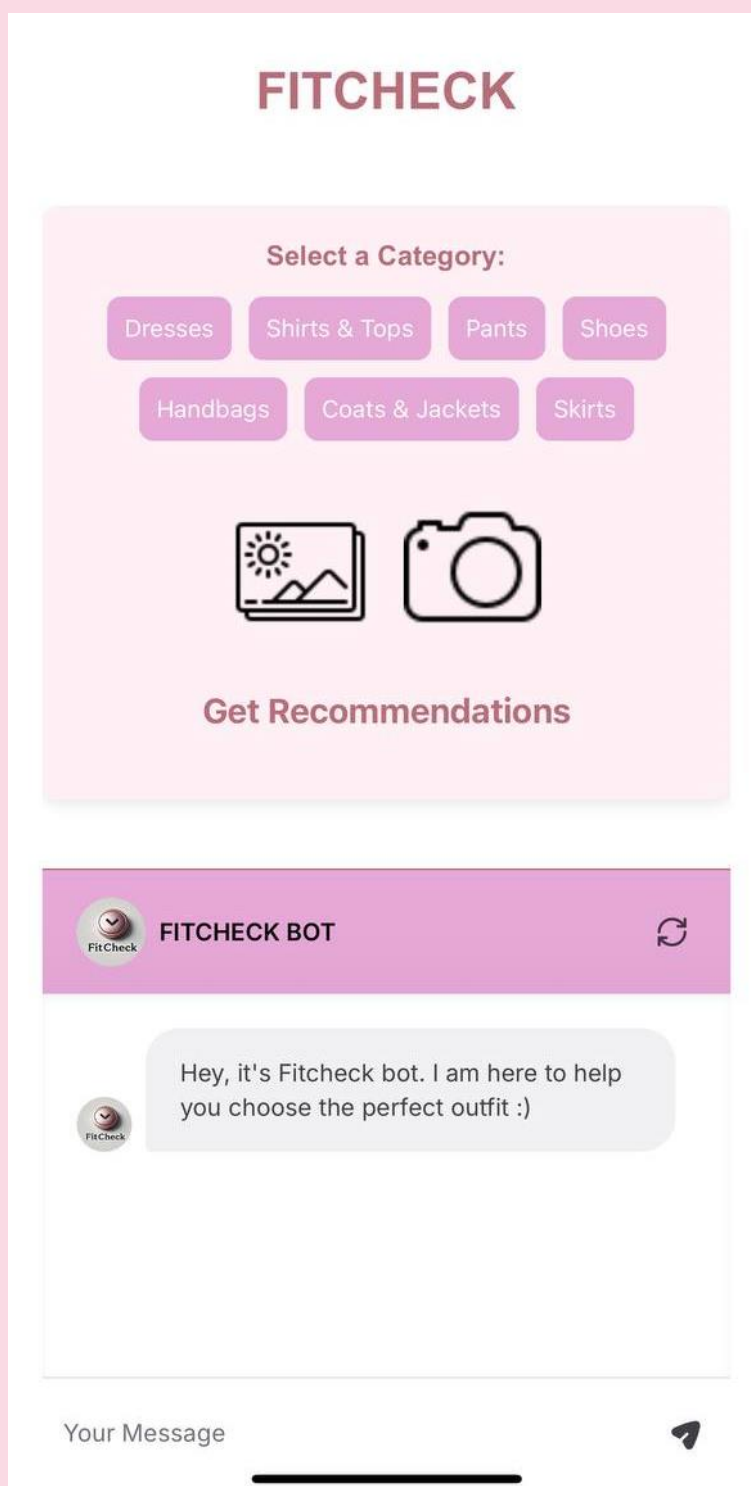
Scarves & Shawls
Product ID: 8963
Compatibility Score: 87.31%



Skirts
Product ID: 17702
Compatibility Score: 87.26%

האפליקציה הסופית

את האפליקציה הסופית יצרנו בעזרת React Native והוספנו אפשרויות של העלאת תמונה מהגלריה או לסרוק את התמונה באמצעות המצלמה. המשתמש בוחר קטגוריה של הפריט אותו רוצה לסרוק, לוחץ על אייקון המלצה לסריקה או על אייקון הגלריה להעלאת תמונה מהגלריה, לאחר מכן לוחץ על כפתור קבלת המלצות ולאחר מכן יועלו לו חמישה פריטי לבוש משלימים. נמחיש את פעולות האפליקציה בעזרת סדרת צילומי מסך:



FITCHECK

Select a Category:

Dresses

Shirts & Tops

Pants

Shoes

Handbags

Coats & Jackets

Skirts



Get Recommendations

FITCHECK



FITCHECK



FITCHECK BOT

בחרנו לשכלל את האפליקציה עם ממשק נוסף שהיא מציעה- בוט מבוסס בינה מלאכותית אשר אומן באמצעות מידע בנושאי אופנה- איתו ניתן להתייעץ בענייני סטיילינג בלבד. נצרף תמונה של התכתבות עם הבוט:

